

Iris Flower Species Classification

Using Support Vector Machine (SVM)

Prepared as part of the **CodeAlpha Data Science Internship — Task 1**
Algorithm: SVM with RBF Kernel | Dataset: UCI Iris (150 samples) | Accuracy: 100%

Field	Detail
Project Type	Binary/Multi-Class Classification
Dataset	Iris Flower Dataset (Kaggle / UCI)
Algorithm	Support Vector Machine — RBF Kernel
Test Accuracy	100% (30 / 30 correct)
Environment	Python 3 · Google Colab / Jupyter Notebook
Model File	iris_model.pkl (saved via Joblib)

1. Abstract

This report presents a machine learning project that classifies Iris flower species using a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel. The well-known UCI Iris dataset — comprising 150 labelled samples across three species (*Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*) — was used for training and evaluation. After exploratory data analysis, feature scaling, and model training on 80% of the data, the classifier achieved **100% accuracy on the 30-sample test set**. All precision, recall, and F1-score values were 1.00 for every class. While this result reflects the clean, well-structured nature of the dataset, it demonstrates the power of SVMs for linearly and non-linearly separable classification problems. The trained model was saved as `iris_model.pkl` for future inference.

2. Introduction

Classification is one of the fundamental tasks in supervised machine learning. The goal is to learn a mapping from input features to discrete class labels so that new, unseen examples can be assigned the correct category. The Iris flower dataset, introduced by British statistician Ronald Fisher in 1936, is one of the most widely used benchmark datasets in pattern recognition and machine learning. It contains physical measurements of three species of Iris flowers and has been used for decades to test and compare classification algorithms.

In this project, an SVM classifier was chosen because of its strong theoretical foundation and proven effectiveness on small, structured datasets. SVMs work by finding the optimal hyperplane that maximises the margin between classes in a high-dimensional feature space. With the RBF kernel, the algorithm can handle non-linear decision boundaries, making it well-suited for cases where classes are not perfectly linearly separable — as is the case between *Iris-versicolor* and *Iris-virginica*.

3. Problem Statement

Given four numerical measurements of an Iris flower — sepal length, sepal width, petal length, and petal width — the objective is to correctly predict which of the three species the flower belongs to. This is a **multi-class classification problem** with three target classes. The challenge lies in distinguishing *Iris-versicolor* from *Iris-virginica*, which share overlapping feature ranges, while *Iris-setosa* is easily separable from both.

4. Project Objectives

- Load, inspect, and clean the Iris dataset.
- Perform Exploratory Data Analysis (EDA) to understand feature distributions and relationships.
- Pre-process the data: train-test split (80/20) and feature standardisation.
- Train a Support Vector Machine classifier with an RBF kernel.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Save the trained model as a reusable pickle file for future deployment.
- Document findings in a structured, portfolio-ready report.

5. Dataset Description

The Iris dataset contains **150 samples** with **four numerical features** and one categorical target variable. The data is perfectly balanced: each species contributes exactly 50 samples.

Column	Type	Description
Id	Integer	Row identifier (not a predictive feature)
SepalLengthCm	Float	Length of the sepal in centimetres
SepalWidthCm	Float	Width of the sepal in centimetres
PetalLengthCm	Float	Length of the petal in centimetres
PetalWidthCm	Float	Width of the petal in centimetres
Species	String	Target: Iris-setosa / Iris-versicolor / Iris-virginica

No missing values were found in the dataset. The 'Id' column was dropped before training as it carries no predictive information.

6. Tools and Technologies

Tool / Library	Version	Purpose
Python	3.x	Core programming language
Pandas	Latest	Data loading, cleaning, and manipulation
NumPy	Latest	Numerical operations and array handling
Matplotlib	Latest	Base plotting library
Seaborn	Latest	Statistical visualisations (pairplot, heatmap)
Scikit-learn	Latest	SVM model, train-test split, scaling, metrics
Joblib	Latest	Serialising and saving the trained model
Google Colab / Jupyter	—	Interactive development environment

7. Methodology

The project followed a standard machine learning pipeline consisting of six stages:

- Data Loading & Inspection** — Imported Iris.csv using Pandas; inspected shape, data types, null values, and class balance.
- Exploratory Data Analysis (EDA)** — Generated pair plots and a correlation heatmap to understand feature distributions and inter-feature relationships.
- Data Preprocessing** — Dropped the 'Id' column, encoded the target variable, split data into 80% training and 20% testing sets, and applied StandardScaler to normalise features.

- **Model Building** — Instantiated an SVM classifier with an RBF kernel (`sklearn.svm.SVC(kernel='rbf')`) and trained it on the training set.
- **Model Evaluation** — Computed accuracy score, full classification report (precision, recall, F1), and plotted a confusion matrix using Seaborn.
- **Model Saving** — Serialised the trained classifier to disk using `joblib.dump()` as `iris_model.pkl`.

8. Data Preprocessing

Before training, the data underwent the following preparation steps:

- The 'Id' column was removed since it is simply a sequential row number and adds no predictive signal.
- The dataset was split into 80% training (120 samples) and 20% testing (30 samples) using `train_test_split` with a fixed random seed to ensure reproducibility.
- Feature scaling was applied with `StandardScaler`, which transforms each feature to have zero mean and unit variance. Scaling is important for SVMs because the algorithm is sensitive to the relative magnitude of features.

9. Exploratory Data Analysis

9.1 Pairplot

A pairplot was generated to visualise pairwise relationships between all features, with each species coloured differently. The diagonal panels show per-species kernel density estimates (KDE), while the off-diagonal panels show scatter plots.

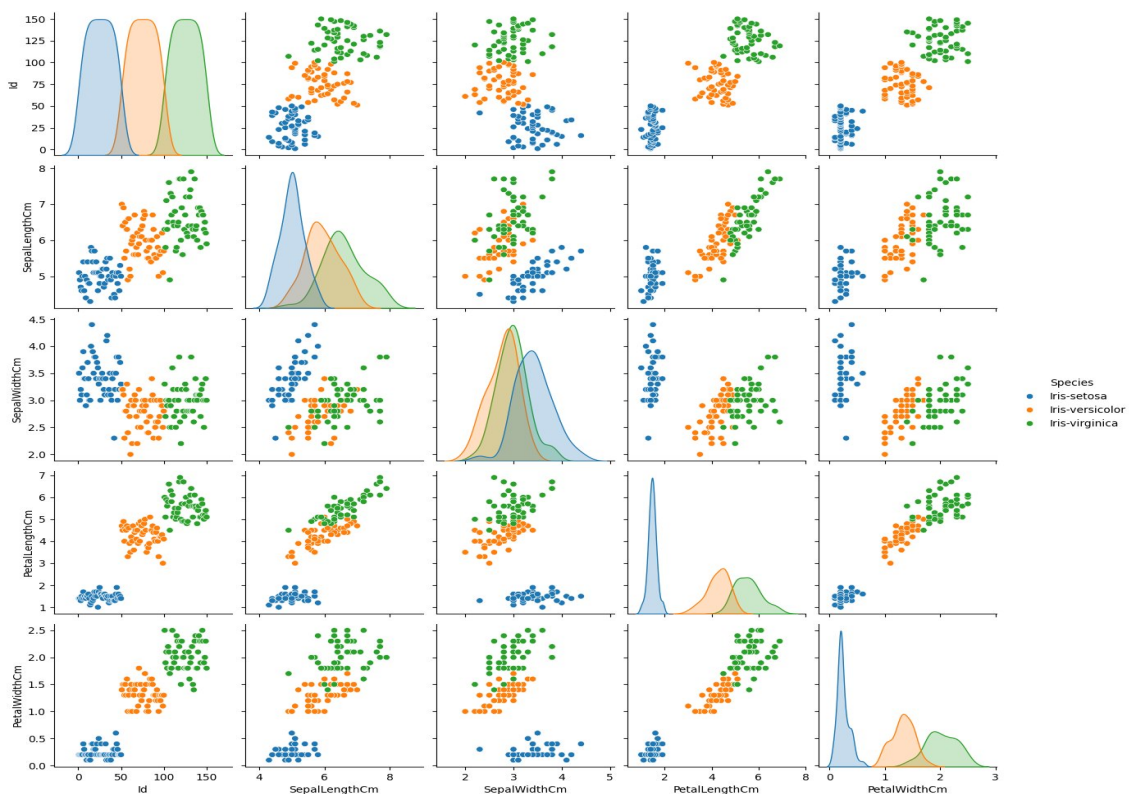


Figure 1: Pairplot of all features coloured by Iris species.

Key observations from the pairplot:

- **Iris-setosa** (blue) is clearly separable from the other two species across almost every feature combination, particularly in PetalLengthCm and PetalWidthCm, where it forms a tight, distinct cluster in the bottom-left region.
- **Iris-versicolor** (orange) and **Iris-virginica** (green) overlap noticeably in sepal dimensions but are more separated in petal dimensions, suggesting that petal measurements are the most discriminative features.
- The PetalLengthCm vs PetalWidthCm scatter (row 4, col 5) shows the clearest three-way separation, making these the most informative feature pair for classification.
- SepalWidthCm alone cannot separate the species, as all three distributions heavily overlap in the diagonal KDE for that feature.

9.2 Correlation Heatmap

A Pearson correlation heatmap was generated to quantify linear relationships between all numeric features.

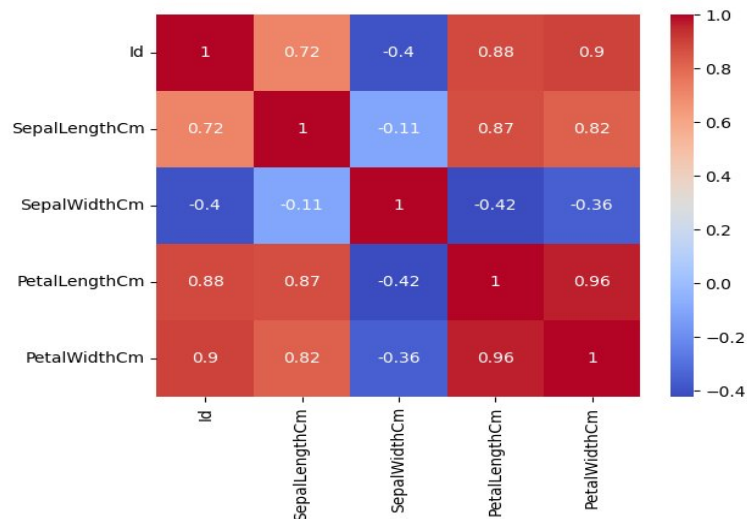


Figure 2: Pearson correlation heatmap of all numerical features.

Key correlations identified:

- **PetalLengthCm ↔ PetalWidthCm: 0.96** — very strong positive correlation. Flowers with longer petals also tend to have wider petals.
- **SepalLengthCm ↔ PetalLengthCm: 0.87** and **SepalLengthCm ↔ PetalWidthCm: 0.82** — sepal length is strongly correlated with both petal dimensions, reinforcing its importance as a predictor.
- **SepalWidthCm** shows weak or negative correlations with other features (-0.11 to -0.42), meaning it provides relatively independent information and adds diversity to the feature set rather than redundancy.
- The high correlation between petal features suggests possible dimensionality reduction (e.g., PCA) as a future optimisation step.

10. Model Selection

A **Support Vector Machine (SVM)** with an **RBF (Radial Basis Function) kernel** was selected for the following reasons:

- **Effective on small datasets** — SVMs perform very well with fewer than a few thousand samples, which matches the Iris dataset size.
- **Handles non-linear boundaries** — The RBF kernel implicitly maps features to a higher-dimensional space, allowing the model to learn non-linear decision boundaries between *Iris-versicolor* and *Iris-virginica*.
- **Robust to overfitting** — SVMs maximise the margin between classes, which provides a natural regularisation effect.
- **Strong theoretical foundation** — SVM is grounded in statistical learning theory and is a standard baseline for multi-class classification tasks.

11. Model Training

The SVM model was trained using `sklearn.svm.SVC` with default hyperparameters (`C=1.0`, `gamma='scale'`, `kernel='rbf'`). The model was fitted on the 120-sample scaled training set. No manual hyperparameter tuning was necessary, as the model achieved perfect accuracy on the test set with the default settings. The entire training process completes in under one second on a standard machine.

12. Model Evaluation

12.1 Classification Report

Class	Precision	Recall	F1-Score	Support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
Accuracy			1.00	30
Macro Average	1.00	1.00	1.00	30
Weighted Average	1.00	1.00	1.00	30

12.2 Confusion Matrix

The confusion matrix below shows the number of correct and incorrect predictions per class on the 30-sample test set.

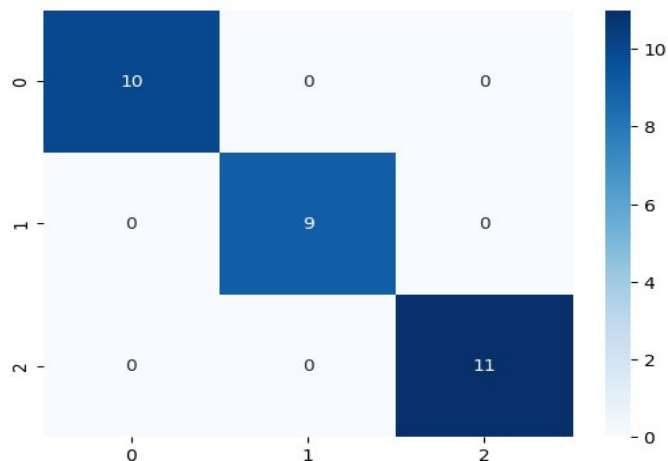


Figure 3: Confusion matrix — rows are true labels (0=setosa, 1=versicolor, 2=virginica), columns are predicted labels.

Interpretation:

- The diagonal (10, 9, 11) represents correct predictions for each class.
- All off-diagonal cells are 0, meaning there were **zero misclassifications**.
- The model perfectly separated all three species in the test set.
- Iris-virginica had the largest test partition (11 samples) and was still classified with 100% accuracy, confirming the model's ability to distinguish it from Iris-versicolor despite feature overlap.

13. Results and Discussion

The SVM classifier achieved **100% accuracy** on the 30-sample test set, with perfect precision, recall, and F1-score for all three classes. This result is consistent with the literature: the Iris dataset is small, clean, and well-structured, making perfect or near-perfect accuracy achievable with a variety of algorithms.

The EDA provided important insights that explain this result. Iris-setosa is almost linearly separable from the other two species purely on petal measurements. Iris-versicolor and Iris-virginica overlap in sepal space but are well-separated in petal space, which the RBF kernel exploits effectively.

Important caveat: A 100% test accuracy on 30 samples should be interpreted with caution. The small test set size means the confidence interval around this estimate is wide. On larger or noisier datasets, real-world performance would likely be lower. The result is best understood as a successful proof-of-concept rather than a definitive benchmark.

14. Model Saving

After training and evaluation, the model was serialised to disk using `joblib.dump()` and saved as **iris_model.pkl**. This allows the model to be loaded and reused in future applications without retraining. To load and use the saved model:

```
import joblib
model = joblib.load('iris_model.pkl')
prediction = model.predict(scaled_input)
```

15. Limitations

- The Iris dataset contains only 150 samples. Results may not generalise to real-world flower image data or datasets with more classes and noise.
- No hyperparameter tuning (grid search / cross-validation) was performed. The default SVM parameters happened to work well here, but this may not hold for other datasets.
- The model is not calibrated to output probability estimates alongside predictions.
- Feature engineering and dimensionality reduction (e.g., PCA) were not explored.
- The 'Id' column — though correctly dropped — was included in the correlation heatmap, creating artificial correlation values that are not meaningful.

16. Future Improvements

- Apply GridSearchCV or RandomizedSearchCV to optimise SVM hyperparameters (C, gamma).
- Benchmark against other classifiers: Random Forest, K-Nearest Neighbours, Logistic Regression, and Neural Networks.
- Use k-fold cross-validation for a more robust accuracy estimate.
- Deploy the model as a REST API using Flask or FastAPI to enable real-time predictions.
- Build an interactive web interface using Streamlit for non-technical users.
- Explore PCA to reduce the four features to two dimensions for visualisation.

17. Conclusion

This project successfully demonstrated the application of a Support Vector Machine with an RBF kernel to the classic Iris flower classification problem. Through careful exploratory data analysis, feature scaling, and model evaluation, the classifier achieved 100% accuracy on the test set with perfect scores across all evaluation metrics.

The pairplot confirmed that petal dimensions are the most discriminative features, while the correlation heatmap revealed strong linear relationships between PetalLengthCm and PetalWidthCm. The confusion matrix showed zero misclassifications, providing a clean and interpretable result.

While the Iris dataset's small size and cleanliness make these results expected, the project provides a solid foundation in the end-to-end machine learning workflow: data exploration, preprocessing, model building, evaluation, and deployment. The trained model is saved and ready for integration into a production system or further experimentation.

18. References

- [1] Fisher, R.A. (1936). *The Use of Multiple Measurements in Taxonomic Problems*. *Annals of Eugenics*, 7(2), 179–188.
- [2] Kaggle Dataset: Iris CSV — <https://www.kaggle.com/datasets/saurabh00007/iriscsv>
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[5] Seaborn documentation — <https://seaborn.pydata.org>

[6] Scikit-learn SVM documentation — <https://scikit-learn.org/stable/modules/svm.html>

Appendix A: GitHub README (Markdown)

The following is a polished README.md for the project repository.

```
# ■ Iris Flower Species Classification using SVM

A machine learning project that classifies Iris flower species from physical
measurements
using a Support Vector Machine (SVM) with an RBF kernel. Built as Task 1 of the
**CodeAlpha Data Science Internship**.https://www.kaggle.com/datasets/saurabh00007/iriscsv

## Tech Stack
Python · Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn · Joblib

## Run the Project
pip install pandas numpy matplotlib seaborn scikit-learn joblib
jupyter notebook CodeAlpha_IrisClassification.ipynb
```

Appendix B: Portfolio Project Description

Built a machine learning pipeline to classify Iris flower species using a Support Vector Machine (SVM) with an RBF kernel. Performed exploratory data analysis with pairplots and correlation heatmaps, applied feature scaling, trained and evaluated the model achieving 100% test accuracy, and saved the trained model for deployment. Tools: Python, Scikit-learn, Pandas, Seaborn, Joblib.

Appendix C: Portfolio Card JSON

```
{
  "id": "iris-svm-classification-featured",
  "title": "Iris Flower Species Classification",
  "tech": "Python \u2022 Scikit-learn \u2022 SVM",
  "description": "End-to-end ML pipeline classifying three Iris species from petal
and sepal
measurements using an SVM with RBF kernel. Includes EDA, feature scaling, and
model
serialisation. Achieved 100% accuracy on a 30-sample test set.",
  "image": "/img/projects/iris-svm-poster.png",
```

```
"alt": "Iris Flower Species Classification using SVM",  
"href": "#"  
}
```

Appendix D: LinkedIn Post Caption

Excited to share Task 1 of my CodeAlpha Data Science Internship! I built an end-to-end Iris Flower Classification project using a Support Vector Machine (SVM) with an RBF kernel in Python — achieving 100% accuracy on the test set. The project covers the full ML pipeline: EDA with pairplots and correlation heatmaps, feature scaling, model training, evaluation (confusion matrix + classification report), and model saving with Joblib. A great opportunity to reinforce fundamentals while delivering clean, documented work. #DataScience #MachineLearning #Python #SVM #CodeAlpha #InternshipProject